

Simulation-two-way

- 1. Sparsity on v (1-SE)
- 2. Smoothness on v
- 3. Smoothness and Sparsity on v
- 4.1. Sparsity on u (1-SE)
- 4.2. Smoothness on u
- 5. Two-way conditional sparsity
- 6. Two-way conditional smoothness
- 7. Conditional two-way sparsity two-way smoothness
- 8. Multivariate hybrid data case

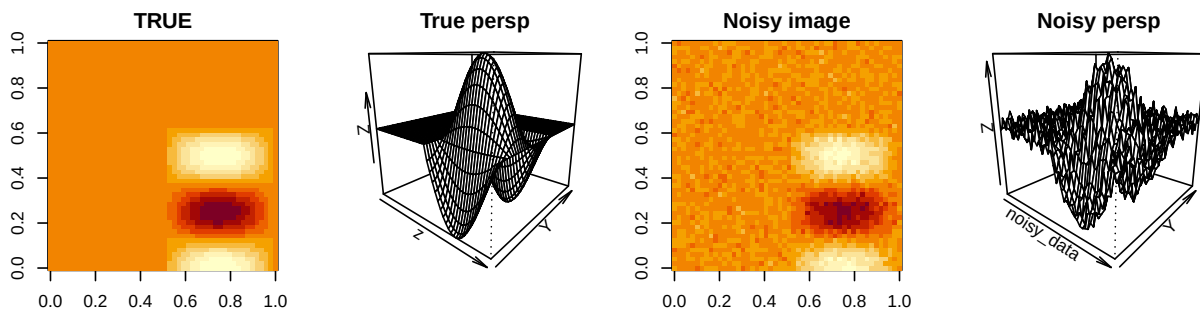
4.1. Sparsity on u (1-SE)

To test Sparsity and Smoothness on u , we need a set of data that has functional behavior on both sides to be able to incorporate regularization on both the left and right singular vectors in the Singular Value Decomposition (SVD) of the data matrix. As a result, we can work with the following data:

```
x <- seq(0,2*pi, len = 50)
y1 <- sin(x); y1[1:26] <- 0
y2 <- cos(2*x); y2[32:50] <- 0
z <- outer(y1,y2)

par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(z, main = "TRUE")
persp(z, theta = 40, main = "True persp")

set.seed(123)
noisy_data <- z + rnorm(length(z), sd = 0.1)
image(noisy_data, main = "Noisy image")
persp(noisy_data, theta = 40, main = "Noisy persp")
```



Let's start with the sparsity on v for this image that we've already tested it!

```
fd_object <- fdClass(data = as.matrix(noisy_data),
                     argval = NULL,
```

```

        Smoothing_parameter = NULL,
        Sparsity_parameter = round(seq(0,49, length.out = 30)))

hd_list <- list(fd_object) # List of fd and rd objects
object_list <- hdClass(hdlist = hd_list,
                       argval = NULL, # Grid points for rows
                       Smoothing_parameter = NULL, # Custom smoothing parameter for rows
                       Sparsity_parameter = round(seq(0,49, length.out = 40))) # Custom sparsity param

# Combine matrices side by side
hd <- object_list
n <- nrow(hd); K_fold <- 5; sparse_tuning_type <- "soft"
n_var <- attr(object_list, "n_var") # Number of variables (# of matrices in object_list)
ncol <- data.frame(attr(object_list, "ncol")) # Number of columns of each matrix

##### Smoothing Parameter #####
# Generate all combinations alphas (one row per combination)
smooth_tuning_col <- expand.grid(attr(object_list, "Smoothing_parameter_col")) # A matrix
smooth_tuning_row <- attr(object_list, "Smoothing_parameter") # A vector

##### level of sparsity (for both functional and non-functional data) #####
sparsity_row_list <- attr(object_list, "Sparsity_parameter")
sparsity_col_list <- attr(object_list, "Sparsity_parameter_col")

X_temp <- hd
# No smoothness list:
S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

cv_sparse_col_SE <- function(data,
                             n_var,
                             ncol,
                             S_alpha_v,
                             S_alpha_u,
                             K_fold,
                             sparse_tuning_result_u,
                             sparse_tuning_result_v,
                             sparse_tuning_type) {

  set.seed(123)
  shuffled_rows <- sample(nrow(data)) # Grouping rows of data matrix
  group_size <- ifelse(round(length(shuffled_rows) / K_fold, digits = 0) == 0,
                      1,
                      round(length(shuffled_rows) / K_fold, digits = 0))

  data_tilde <- data
  fold_errors <- numeric(K_fold) # store errors

  for (k in 1:K_fold) {
    rows_to_remove <- shuffled_rows[((k - 1) * group_size + 1):min(k * group_size, length(shuffled_rows)

```

```

data_train <- data.frame(data_tilde[-rows_to_remove,])
data_test  <- data.frame(matrix(data_tilde[rows_to_remove,], nrow = length(rows_to_remove)))

# Power Algorithm
power_train <- power_algo(data = data_train,
                          n_var = n_var,
                          ncol = ncol,
                          sparse_tuning_result_u = sparse_tuning_result_u,
                          sparse_tuning_result_v = sparse_tuning_result_v,
                          S_alpha_v = S_alpha_v,
                          S_alpha_u = S_alpha_u[-rows_to_remove, -rows_to_remove],
                          sparse_tuning_type = sparse_tuning_type)

v_train <- power_train[[1]]
u_test  <- as.matrix(data_test) %*% as.matrix(v_train)

# Fold-wise error
fold_errors[k] <- sum((data_test - u_test %*% t(v_train))^2) / nrow(data_test)
}
CV_mean <- mean(fold_errors)
CV_se   <- sd(fold_errors) / sqrt(K_fold)

return(list(CV_error = CV_mean, SE = CV_se, fold_errors = fold_errors))
}

# Tuning across sparsity levels using 1-SE rule
CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- list()
CV_results <- list()

gamma_v <- rep(0, n_var)
for (i in 1:n_var) {
  gamma_Xi <- sparsity_col_list[[i]]
  cv_means <- cv_ses <- fold_errors_vec <- numeric(length(gamma_Xi))

  for (j in seq_along(gamma_Xi)) {
    gamma_v[i] <- gamma_Xi[j]

    cv_result <- cv_sparse_col_SE(data = X_temp,
                                n_var = n_var,
                                ncol = ncol,
                                S_alpha_v = S_alpha_v,
                                S_alpha_u = diag(nrow(X_temp)),
                                K_fold = K_fold,
                                sparse_tuning_result_u = 0,
                                sparse_tuning_result_v = gamma_v,
                                sparse_tuning_type = sparse_tuning_type)

    cv_means[j] <- cv_result$CV_error
    cv_ses[j] <- cv_result$SE
    #fold_errors_vec <- cv_result$fold_errors
  }
}

```

```

CV_results[[i]] <- data.frame(gamma_Xi, cv_means, cv_ses)

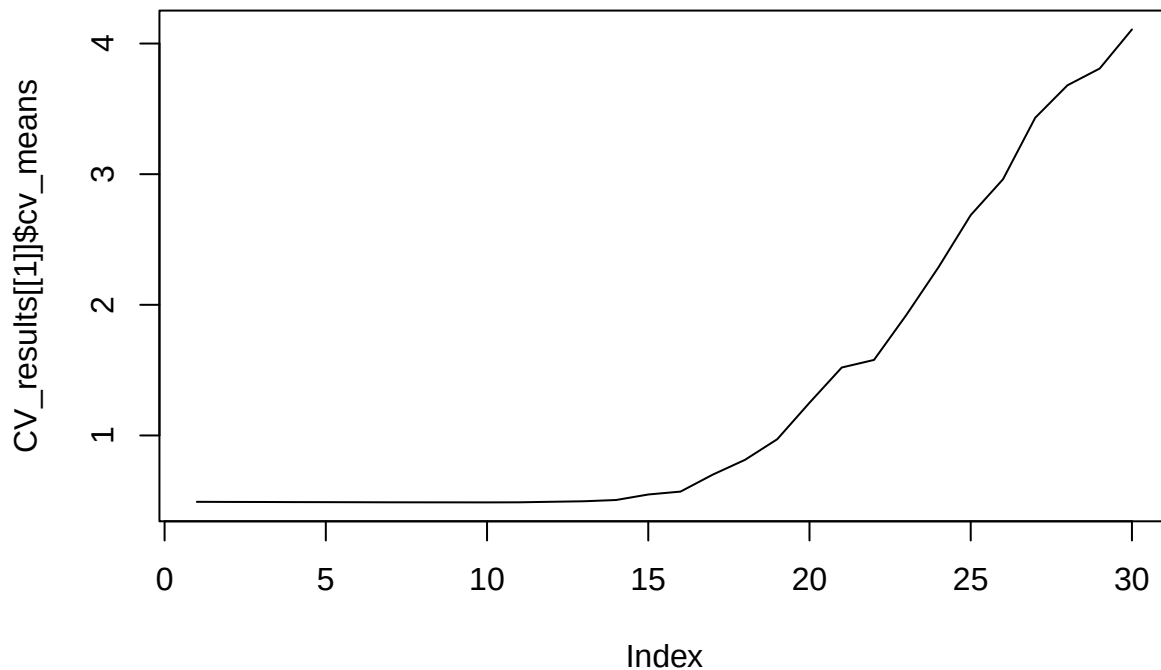
# 1-SE rule
j_min <- which.min(cv_means)
threshold <- cv_means[j_min] + cv_ses[j_min]
j_1se <- max(which(cv_means <= threshold)) # Most sparse within 1-SE

# Store results
CV_scores_result_v[[i]] <- list(errors = cv_means, SEs = cv_ses)
sparse_tuning_selection_v <- gamma_Xi[j_1se]
gamma_v[i] <- sparse_tuning_selection_v
}
gamma_v

```

```
## [1] 19
```

```
plot(CV_results[[1]]$cv_means, type = 'l')
```



```

par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,

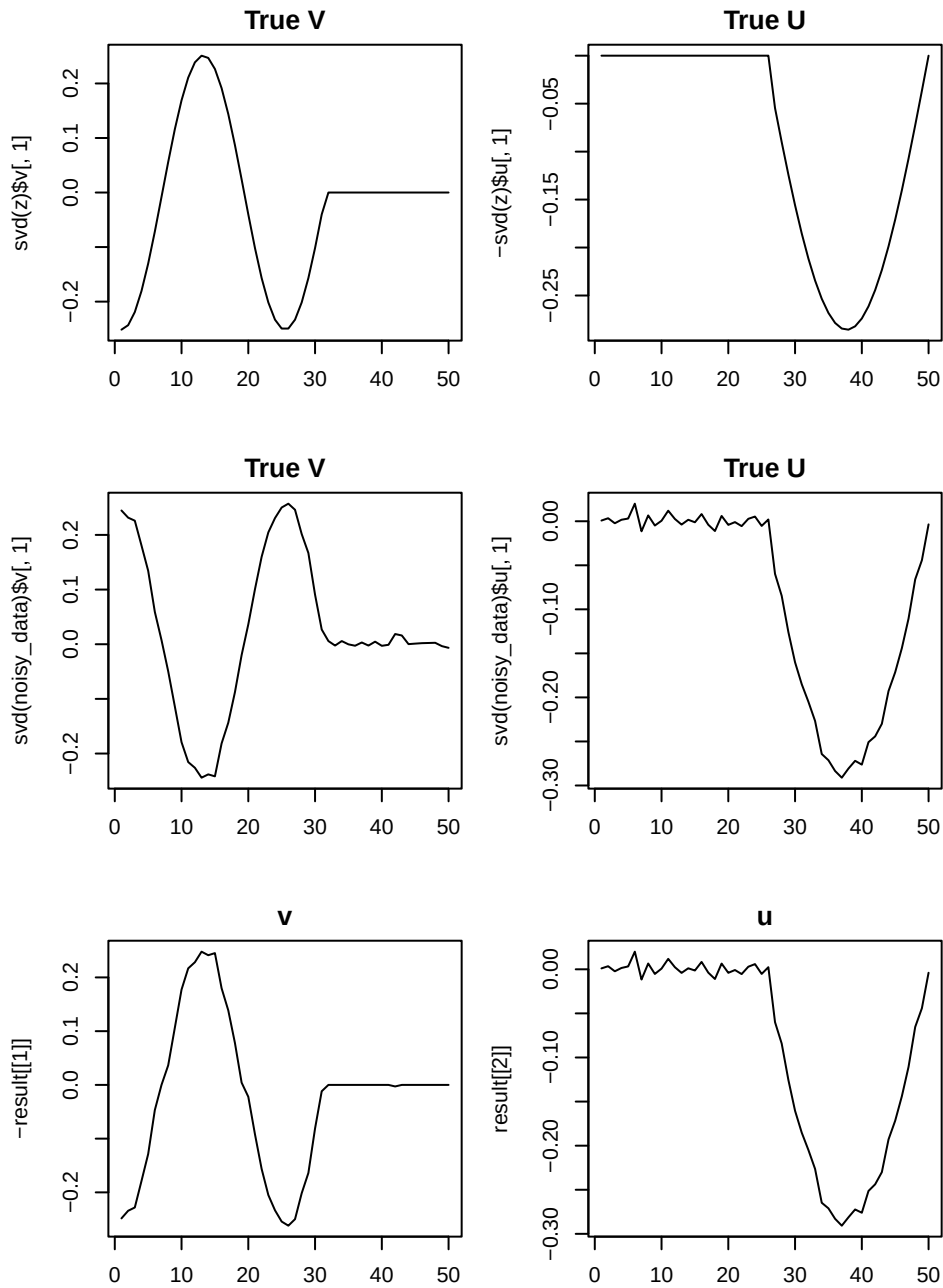
```

```

ncol = ncol,
sparse_tuning_result_u = 0, # A fixed number
sparse_tuning_result_v = gamma_v, # A vector (length = p)
S_alpha_v = S_alpha_v, # A list (length = p)
S_alpha_u = diag(nrow(X_temp)), # A matrix
sparse_tuning_type= sparse_tuning_type)

matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```

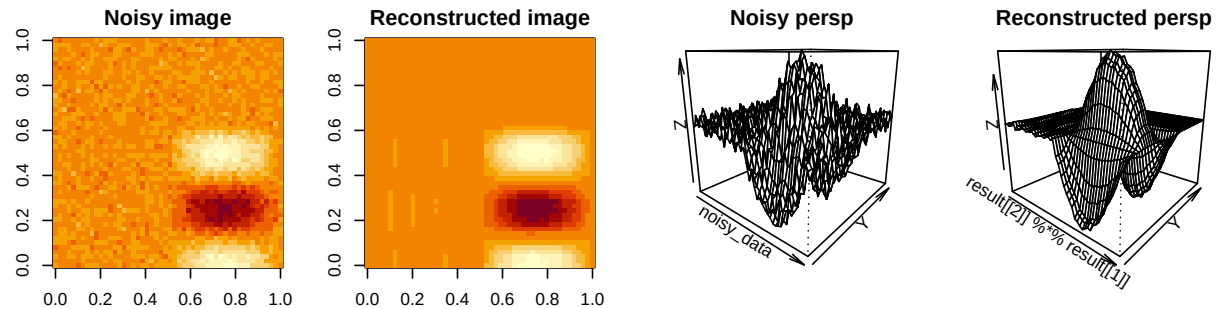


```

par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")

```



Based on the images shown above, level of sparsity for u needs to be 25. Now, we can implement sparsity on u :

```

sparse_tuning_u <- attr(object_list, "Sparsity_parameter")

S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- c()
result = c()

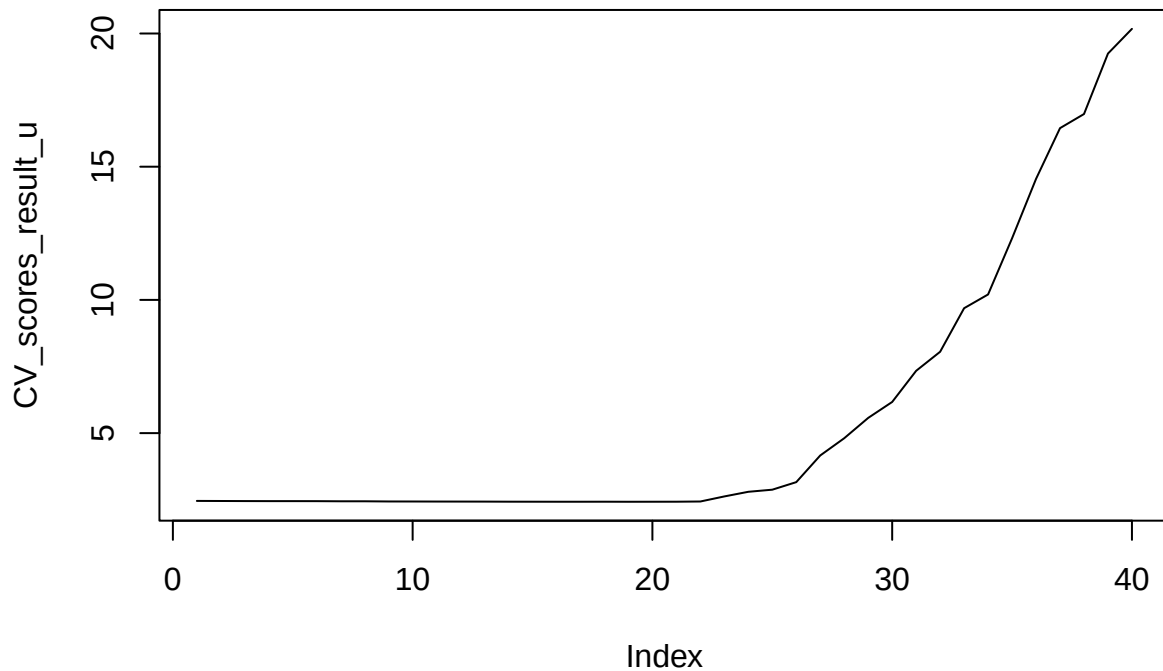
# Step 1: Sparsity on u with no smoothness
for (sparse_tuning_single in sparse_tuning_u) {
  sparse_score = cv_sparse_row(data=X_temp,
                              ncol = ncol,
                              n_var = n_var,
                              S_alpha_v = S_alpha_v,
                              S_alpha_u = diag(nrow(X_temp)),
                              K_fold = K_fold,
                              sparse_tuning_result_u = sparse_tuning_single,
                              sparse_tuning_result_v = rep(0, n_var),
                              sparse_tuning_type = sparse_tuning_type)

  CV_scores_result_u <- c(CV_scores_result_u, sparse_score)
  if (sparse_score <= CV_score_sparse_u) {
    CV_score_sparse_u = sparse_score
    gamma_u = sparse_tuning_single
  }
}
gamma_u

```

```
## [1] 23
```

```
plot(CV_scores_result_u, type = 'l')
```

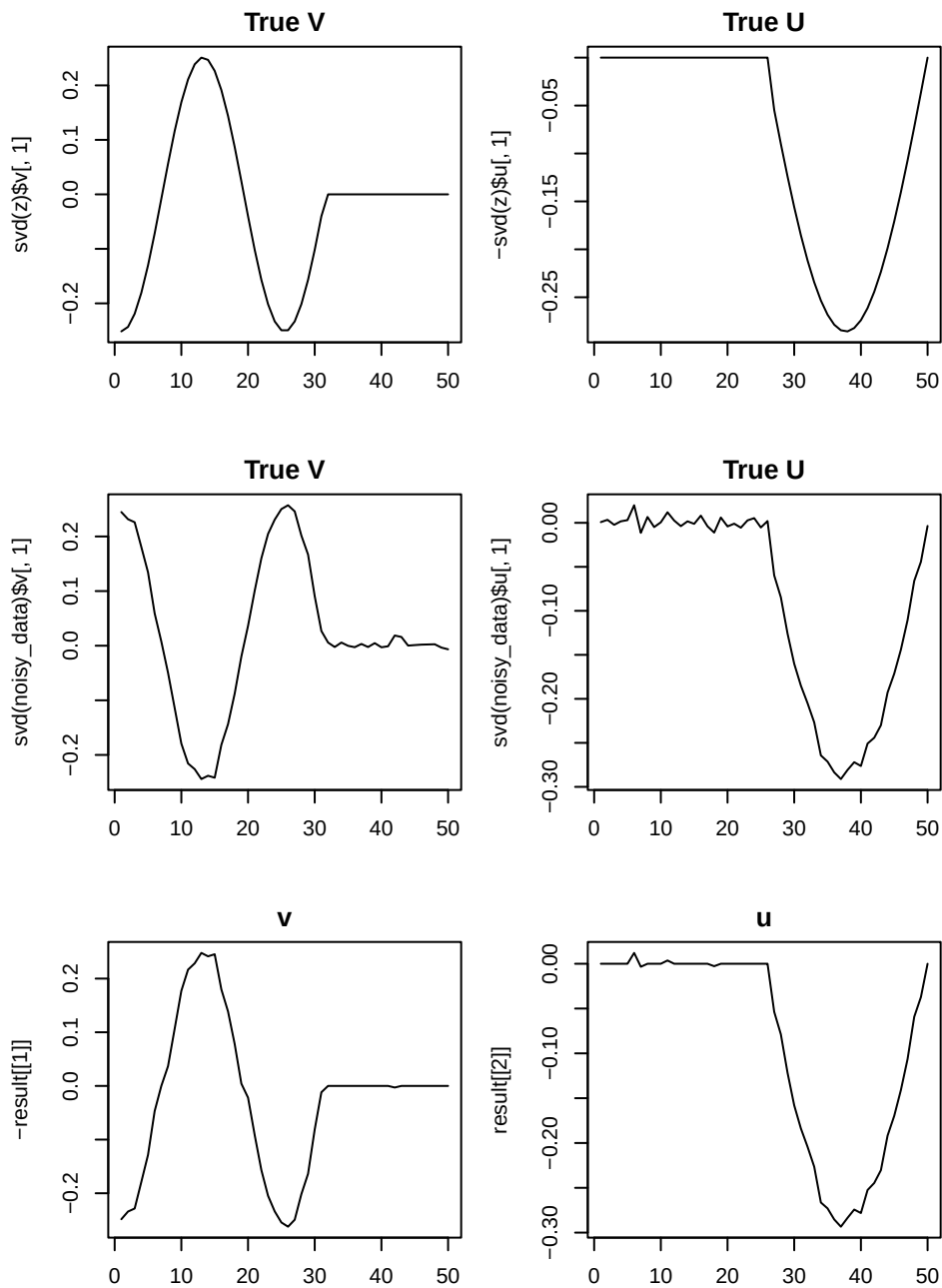


```
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

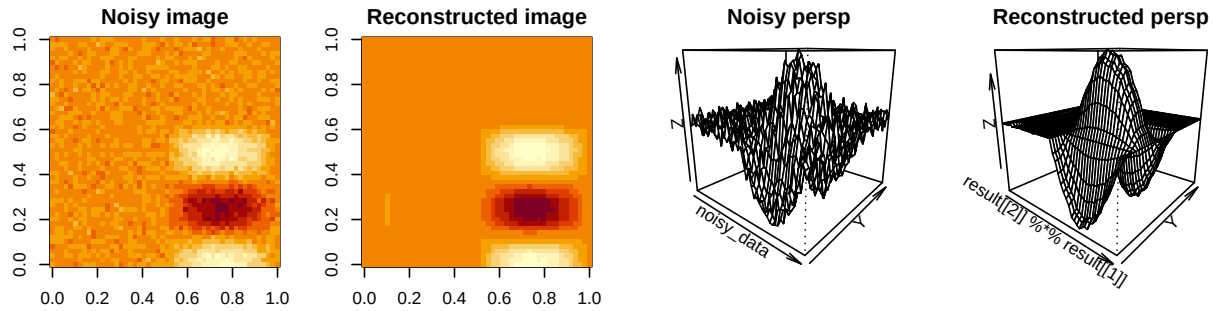
result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = gamma_u, # A fixed number
                     sparse_tuning_result_v = gamma_v, # A vector (length = p)
                     S_alpha_v = S_alpha_v, # A list (length = p)
                     S_alpha_u = diag(nrow(X_temp)), # A matrix
                     sparse_tuning_type = sparse_tuning_type)

matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')
```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```

Now, our goal is to add 1 SE to the CV formula:

```
cv_sparse_row_SE <- function(data,
                             n_var,
                             ncol,
                             S_alpha_u,
                             K_fold,
                             sparse_tuning_result_u, # vector of candidate sparsity levels for rows (u)
                             sparse_tuning_result_v, # fixed right sparsity (vector)
                             sparse_tuning_type) {

  set.seed(123)
  shuffled_cols <- sample(ncol(data)) # Shuffle column indices
  group_size <- ceiling(length(shuffled_cols) / K_fold)
  m <- ncol(data)

  CV_errors <- numeric(length(sparse_tuning_result_u))
  SE_errors <- numeric(length(sparse_tuning_result_u))
  fold_errors_list <- vector("list", length(sparse_tuning_result_u))

  for (j in 1:length(sparse_tuning_result_u)) {
    gamma_u <- sparsity_row_list[j]
    fold_errors <- numeric(K_fold)

    for (k in 1:K_fold) {
      # Define indices for test columns
      cols_to_remove <- shuffled_cols[((k - 1) * group_size + 1):min(k * group_size, length(shuffled_cols))]

      # Create training and test sets
      data_train <- data[, -cols_to_remove]
      data_test <- data[, cols_to_remove]

      # Update column structure
      updated_ncol <- update_ncol(ncol, cols_to_remove)

      # Create identity smoother matrices (no smoothing)
      S_alpha_v_train <- lapply(1:n_var, function(i) diag(updated_ncol[, i]))

      # Run power algorithm
      power_result <- power_algo(data = data_train,
                                n_var = n_var,
                                ncol = updated_ncol,
                                sparse_tuning_result_u = gamma_u,
                                sparse_tuning_result_v = sparse_tuning_result_v,
```

```

        S_alpha_v = S_alpha_v_train,
        S_alpha_u = S_alpha_u,
        sparse_tuning_type = sparse_tuning_type)

    u_hat <- power_result[[2]]
    v_test <- t(as.matrix(data_test)) %*% u_hat

    # Compute reconstruction error for this fold
    fold_errors[k] <- sum((data_test - u_hat %*% t(v_test))^2) / ncol(data_test)
  }

  # Compute CV error and standard error for current gamma
  CV_errors[j] <- mean(fold_errors)
  SE_errors[j] <- sd(fold_errors) / sqrt(K_fold)
  fold_errors_list[[j]] <- fold_errors
}

# One-standard-error rule selection
j_star <- which.min(CV_errors)
CV_1se_threshold <- CV_errors[j_star] + SE_errors[j_star]
j_1se <- max(which(CV_errors <= CV_1se_threshold)) # most regularized (largest gamma)
gamma_u <- sparse_tuning_result_u[j_1se]

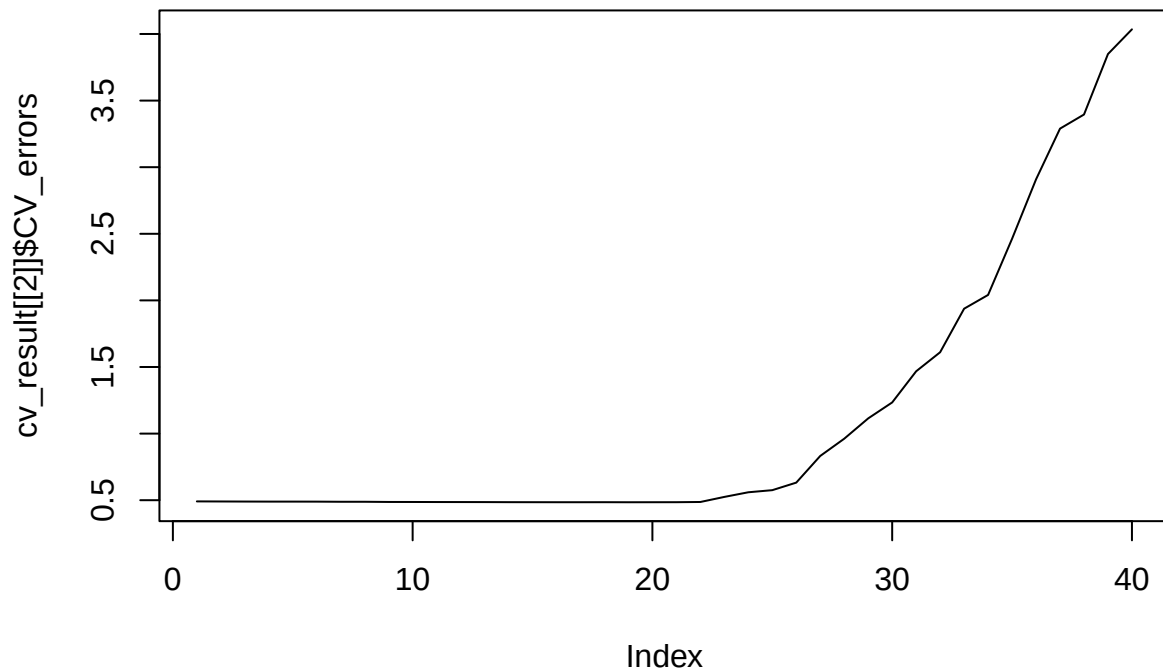
return(list(gamma_u, data.frame(sparse_tuning_result_u, CV_errors, SE_errors)))
}

cv_result <- cv_sparse_row_SE(data = X_temp,
                             n_var = n_var,
                             ncol = ncol,
                             S_alpha_u = diag(nrow(X_temp)),
                             K_fold = K_fold,
                             sparse_tuning_result_u = sparsity_row_list,
                             sparse_tuning_result_v = rep(0, n_var),
                             sparse_tuning_type = sparse_tuning_type)

gamma_u <- cv_result[[1]]
gamma_u

## [1] 26
plot(cv_result[[2]]$CV_errors, type = 'l')

```



```

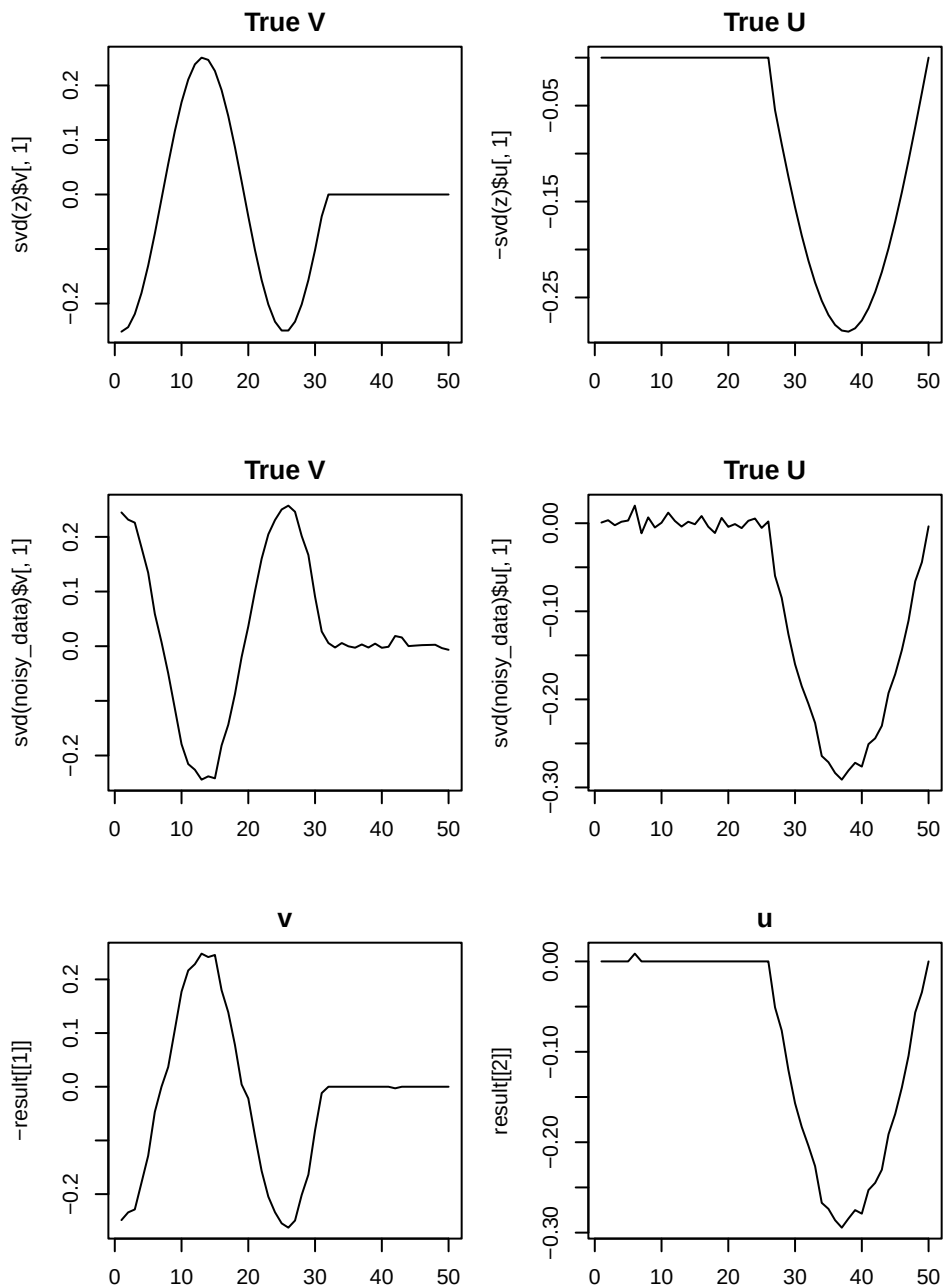
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = gamma_u, # A fixed number
                     sparse_tuning_result_v = gamma_v, # A vector (length = p)
                     S_alpha_v = S_alpha_v, # A list (length = p)
                     S_alpha_u = diag(nrow(X_temp)), # A matrix
                     sparse_tuning_type = sparse_tuning_type)

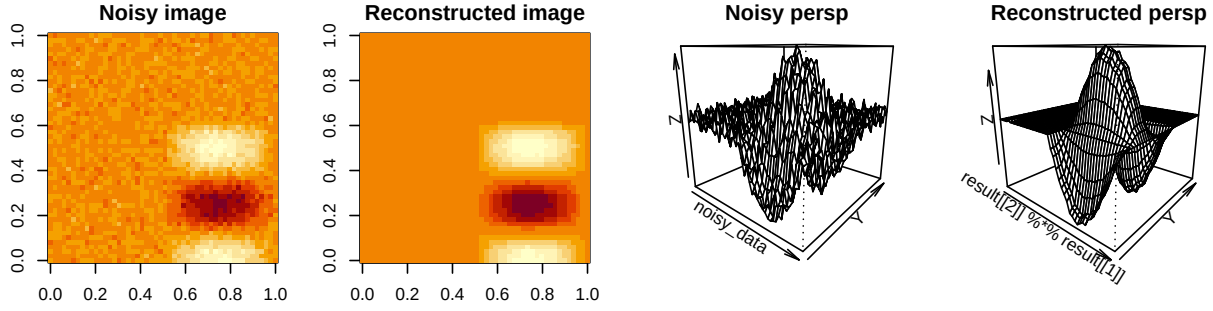
matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```



4.2. Smoothness on u

Let's start by implementing smoothness on v using the following formula:

$$\text{GCV}_v(\alpha_v; \alpha_u = 0) = \frac{1}{m} \sum_{k=1}^K \frac{\left\| \frac{X_k^T u}{\|u\|^2} - v_k \right\|^2}{\left(1 - \frac{1}{m_k} \cdot \frac{\text{tr}(S_v(\alpha_k))}{1 + \alpha_u \cdot \mathcal{R}_u(u)} \right)^2}$$

```
smoothness_type <- "Second_order"
smooth_tuning_col <- expand.grid(attr(object_list, "Smoothing_parameter_col")) # A matrix

##### Grid Points #####
GridPoints_u <- attr(object_list, "GridPoints_u")
GridPoints_v <- attr(object_list, "GridPoints_v")

##### S_alpha for all alphas #####
S_alpha_list_u <- S_alpha_list_v <- Omegas_v <- list()
index <- 0
cat("Preprocessing ... \n")

## Preprocessing ...

n_iter1 <- nrow(smooth_tuning_col) # The number of alphas
pb <- txtProgressBar(min = 0,      # Minimum value of the progress bar
                    max = n_iter1, # Maximum value of the progress bar
                    style = 3,     # Progress bar style (also available style = 1 and style = 2)
                    width = 50,    # Progress bar width. Defaults to getOption("width")
                    char = "=")    # Character used to create the bar

## |

# S_alpha for v
for (alpha_index in 1:nrow(smooth_tuning_col)) {
  index <- index + 1
  S_u <- S_v <- Omega_v <- Omegas_u <- list()
  for (i in 1:n_var) {
    alpha <- as.numeric(smooth_tuning_col[alpha_index, i])
    if (is.null(GridPoints_v[[i]])) {
      S_v[[i]] <- diag(ncol[, i])
    } else {
      get.pen.result <- get.pen(td = GridPoints_v[[i]],
                                alpha = alpha,
                                type = smoothness_type)
      S_v[[i]] <- get.pen.result$S.alpha
      Omega_v[[i]] <- get.pen.result$Omega
    }
  }
}
```

```

    }
  }
  S_alpha_list_v[[index]] <- S_v #as.matrix(bdiag(S_v))
  Omegas_v[[index]] <- Omega_v
  setTxtProgressBar(pb, index)
}

##      |=====

# S_alpha for u
if(!is.null(smooth_tuning_row)){
  for (alpha_index in 1:length(smooth_tuning_row)) {
    alpha <- as.numeric(smooth_tuning_row[alpha_index])
    get.pen.result <- get.pen(td = GridPoints_u,
                             alpha = alpha,
                             type = smoothness_type)

    S_alpha_list_u[[alpha_index]] <- get.pen.result$S.alpha
    Omegas_u[[alpha_index]] <- get.pen.result$Omega
  }
}

library(Matrix)
power_algo2 <- function(data,
                        n_var,
                        ncol,
                        sparse_tuning_result_u,
                        sparse_tuning_result_v,
                        S_alpha_v,
                        S_alpha_u,
                        sparse_tuning_type,
                        conditional = FALSE,
                        alpha_Omega_v,
                        alpha_Omega_u) {
  rownames(data) <- NULL; colnames(data) <- NULL
  data <- as.matrix(data)

  start_col <- 1; splitted_data <- list()
  for (i in 1:n_var) {
    num_cols <- ncol[[i]]
    splitted_data[[i]] <- data[, start_col:(start_col + num_cols - 1)]
    start_col <- start_col + num_cols
  }

  v_old <- svd(data)$v[, 1]
  errors <- Inf; thresh <- 1e-10

  while (errors > thresh) {
    # Compute u
    Xv <- data %*% v_old
    u_raw <- sparse_pen_fun(y = Xv,
                           tuning_parameter = sparse_tuning_result_u,
                           type = sparse_tuning_type)
  }
}

```

```

if (all(u_raw == 0)) u_raw <- Xv

if (conditional) {
  v_old <- as.vector(v_old)
  alpha_Omega_v <- as.matrix(alpha_Omega_v)
  storage.mode(alpha_Omega_v) <- "numeric"
  denom_u <- as.numeric(t(v_old) %*% (diag(nrow(alpha_Omega_v)) + alpha_Omega_v) %*% v_old)
  u_new <- S_alpha_u %*% u_raw / denom_u
} else {
  u_new <- S_alpha_u %*% u_raw
}

# Compute v
v_new <- c()
for (i in 1:n_var) {
  sparse_param <- as.integer(sparse_tuning_result_v[i])
  Xu_i <- t(splitted_data[[i]]) %*% u_new
  v_part_raw <- sparse_pen_fun(y = Xu_i,
                             tuning_parameter = sparse_param,
                             type = sparse_tuning_type)
  if (all(v_part_raw == 0)) v_part_raw <- Xu_i

  if (conditional) {
    norm_u2 <- as.numeric(norm_vec(u_new)^2)
    R_u <- as.numeric(t(u_new) %*% alpha_Omega_u %*% u_new / norm_u2)
    denom_v <- 1 + R_u
    v_part <- S_alpha_v[[i]] %*% v_part_raw / denom_v
  } else {
    v_part <- S_alpha_v[[i]] %*% v_part_raw
  }

  v_new <- c(v_new, v_part)
}

v_new <- v_new / norm_vec(v_new)
errors <- sum((v_new - v_old)^2)
v_old <- v_new
}

if (!conditional) u_new <- u_new / norm_vec(u_new)
return(list(v_new, u_new))
}

opt_alpha_v <- function(X,
                       n_var,
                       ncol,
                       S_alphas_v, # List of length = n_iter, each element is list of length = n_var
                       S_alphas_u, # Matrix (n x n)
                       alphas_v,   # Matrix of alpha_v values (n_iter x n_var)
                       alpha_u,    # scalar
                       Omega_u,    # A matrix associated with alpha_u

```

```

        Omegas_v,      # List of length = n_iter, each element is list of length = n_var
        sparse_tuning_result_u,
        sparse_tuning_result_v,
        sparse_tuning_type) {

n_iter <- nrow(alphas_v)
n <- nrow(X)
m <- ncol(X)
GCV <- numeric(n_iter)

if (all(alphas_v == 0)) {
  return(list(GCV_v = Inf,
             opt.alpha_v = rep(0, n_var),
             opt_s.alpha_v = replicate(n_var, diag(m / n_var), simplify = FALSE),
             GCVdf_v = data.frame(alphas_v, rep(Inf, n_iter))))
} else {
  alpha_Omega_u <- alpha_u * Omega_u
  for (i in 1:n_iter) {
    S_alpha_v <- S_alphas_v[[i]]
    Omega_v <- Omegas_v[[i]]
    alpha_Omega_v <- as.matrix(bdiag(lapply(1:n_var, function(j) alphas_v[i, j] * Omega_v[[j]])))

    # Power Algorithm
    power_result <- power_algo2(data = X,
                                n_var = n_var,
                                ncol = ncol,
                                sparse_tuning_result_u = sparse_tuning_result_u,
                                sparse_tuning_result_v = sparse_tuning_result_v,
                                S_alpha_v = S_alpha_v,
                                S_alpha_u = S_alphas_u,
                                sparse_tuning_type = sparse_tuning_type,
                                conditional = TRUE,
                                alpha_Omega_v = alpha_Omega_v,
                                alpha_Omega_u = alpha_Omega_u)

    v <- power_result[[1]]
    u <- power_result[[2]]

    norm_u2 <- as.numeric(norm_vec(u)^2)
    Xu_proj <- as.vector(t(X) %*% u / norm_u2)
    R_u <- as.numeric(t(u) %*% Omega_u %*% u / norm_u2)

    GCV_alpha <- 0
    idx <- 0
    for (k in 1:n_var) {
      m_k <- as.integer(ncol[k])
      v_k <- v[(idx + 1):(idx + m_k)]
      xuk <- Xu_proj[(idx + 1):(idx + m_k)]
      S_k <- S_alpha_v[[k]]

      trace_k <- sum(diag(S_k))
      denom_k <- (1 - (1 / m_k) * (trace_k / (1 + alpha_u * R_u)))^2
      numer_k <- norm_vec(xuk - v_k)^2

```



```

        GCV_alpha <- GCV_alpha + numer_k / denom_k
        idx <- idx + m_k
    }

    GCV[i] <- GCV_alpha / m
}

opt.alpha <- alphas_v[which.min(GCV), ]
opt_s.alpha <- S_alphas_v[[which.min(GCV)]]

return(list(GCV_v = GCV,
            opt.alpha_v = opt.alpha,
            opt_s.alpha_v = opt_s.alpha,
            GCVdf_v = data.frame(alphas_v, GCV)))
}
}

GCV_result_v = opt_alpha_v(X = X_temp,
                           n_var = n_var,
                           ncol = ncol,
                           S_alphas_v = S_alpha_list_v, # A list (length = n_var)
                           S_alphas_u = diag(nrow(X_temp)),
                           alphas_v = smooth_tuning_col,
                           alpha_u = 0,
                           Omega_u = diag(nrow(X_temp)),
                           Omegas_v = Omegas_v,
                           sparse_tuning_result_u = 0,
                           sparse_tuning_result_v = c(0,0),
                           sparse_tuning_type = sparse_tuning_type)

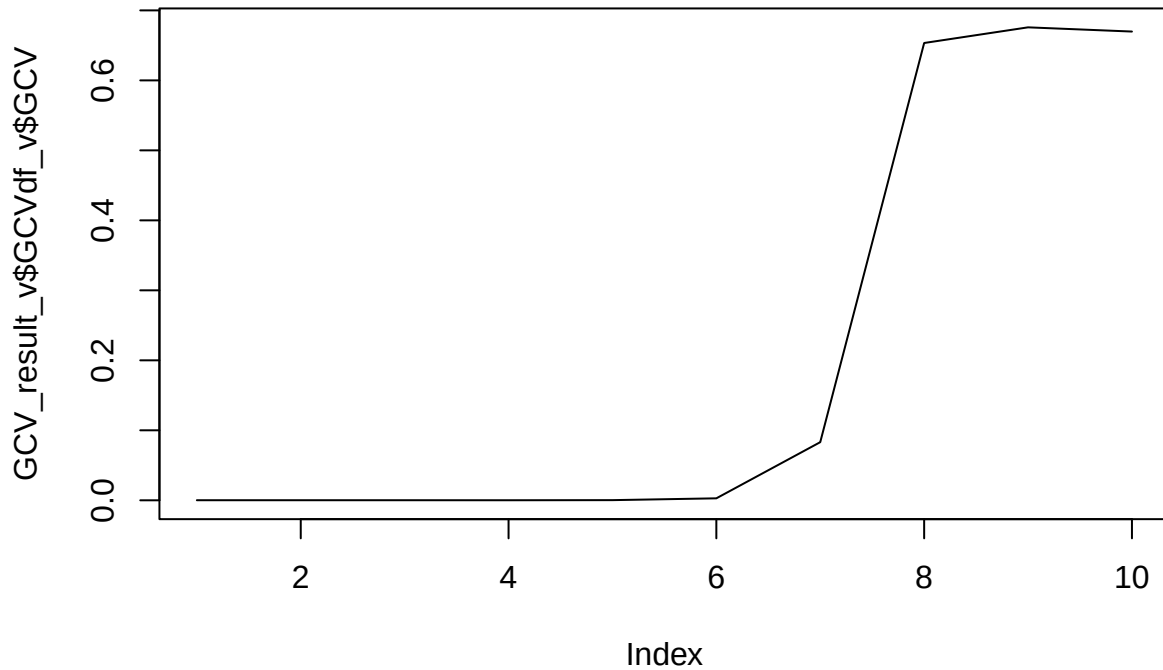
GCV_result_v$opt.alpha_v

## [1] 3.027727e-06
GCV_result_v$GCVdf_v

##           Var1           GCV
## 1  9.313226e-10 1.046472e-04
## 2  1.379661e-08 1.030858e-04
## 3  2.043829e-07 8.979404e-05
## 4  3.027727e-06 8.135892e-05
## 5  4.485274e-05 2.027030e-04
## 6  6.644482e-04 2.818308e-03
## 7  9.843133e-03 8.305688e-02
## 8  1.458161e-01 6.534116e-01
## 9  2.160119e+00 6.757592e-01
## 10 3.200000e+01 6.697427e-01

plot(GCV_result_v$GCVdf_v$GCV, type = 'l')

```



```

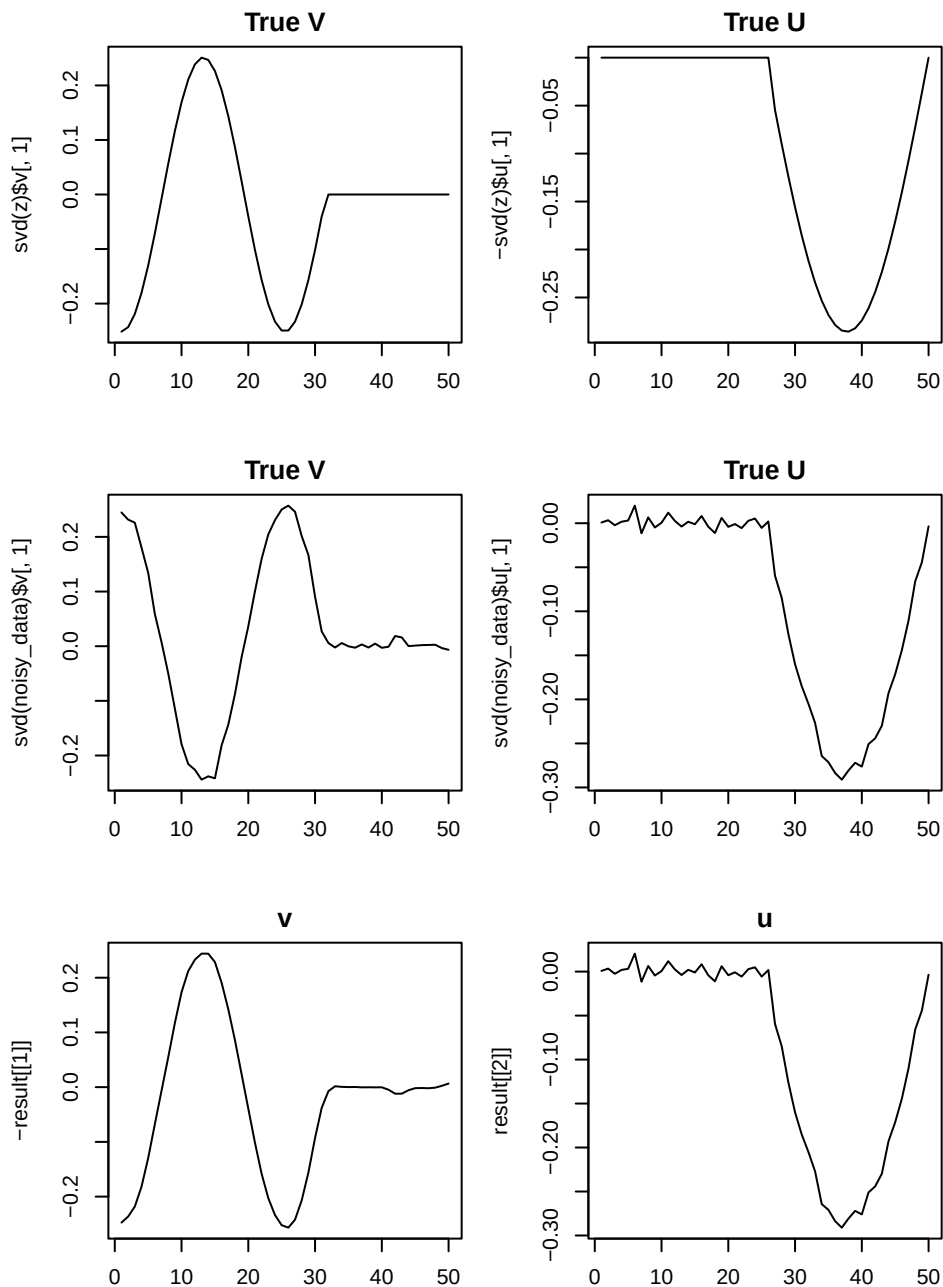
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = 0, # A fixed number
                     sparse_tuning_result_v = c(0,0), # A vector (length = p)
                     S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (length = p)
                     S_alpha_u = diag(nrow(X_temp)), # A matrix
                     sparse_tuning_type = sparse_tuning_type)

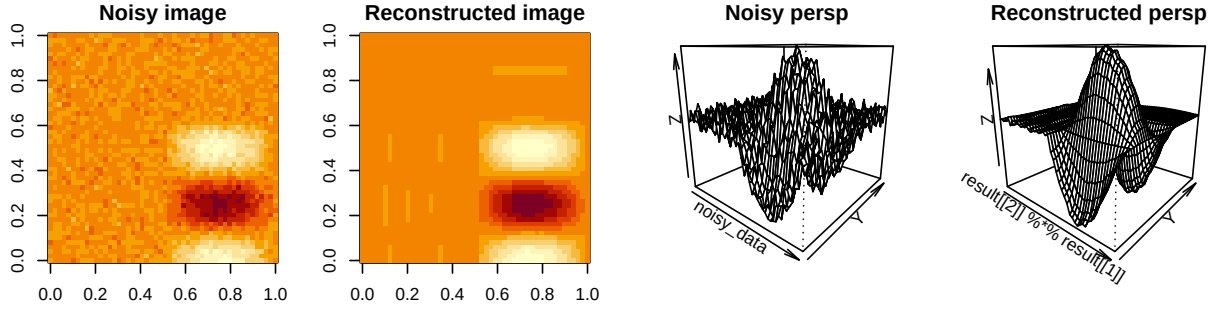
matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```



And now, we're going to use the following formula to get optimal α_u :

$$\text{GCV}_u(\alpha_u; \alpha_v) = \frac{\frac{1}{n} \left\| \frac{\mathbf{X}v}{\|v\|^2} - u \right\|^2}{\left(1 - \frac{1}{n} \frac{\text{tr}(\mathbf{S}_u(\alpha_u))}{1 + \alpha_v \mathbf{R}_v(v)} \right)^2}$$

```

opt_alpha_u <- function(X,
  n_var,
  ncol,
  S_alphas_v, # A list (length = n_var)
  S_alphas_u, # A list (length = length(alphas))
  alphas_u,   # A vector of alpha_u
  alpha_v,    # scalar
  Omega_v,    # A matrix associated with alpha_v
  Omegas_u,   # List of length, each element is matrix
  sparse_tuning_result_u,
  sparse_tuning_result_v,
  sparse_tuning_type) {

  n_iter <- length(alphas_u)
  n <- nrow(X)
  GCV <- numeric(n_iter)

  if (all(alphas_u == 0)) {
    return(list(GCV_u = Inf, opt.alpha_u = 0, opt_s.alpha_u = diag(n), GCVdf_u = data.frame(alphas_u, r
  } else {
    alpha_Omega_v <- alpha_v * Omega_v
    for (i in 1:n_iter) {
      GCV_alpha <- 0
      S <- S_alphas_u[[i]]
      Omega_u <- Omegas_u[[i]]
      alpha_Omega_u <- as.matrix(alphas_u[i] * Omega_u)

      power_result <- power_algo2(data = X,
        n_var,
        ncol,
        sparse_tuning_result_u, # A fixed number
        sparse_tuning_result_v, # A vector (length = p)
        S_alpha_v = list(S_alphas_v), # A list (length = p)
        S_alpha_u = S, # A matrix
        conditional = TRUE,
        alpha_Omega_v = alpha_Omega_v,
        alpha_Omega_u = alpha_Omega_u,

```

```

                                sparse_tuning_type = sparse_tuning_type)

  u <- power_result[[2]]
  v <- t(X) %*% u
  v <- as.vector(v)
  Omega_v <- as.matrix(Omega_v)
  storage.mode(Omega_v) <- "numeric"

  norm_v2 <- as.numeric(norm_vec(v)^2)
  R_v <- as.numeric(t(v) %*% Omega_v %*% v / norm_v2)

  GCV_alpha <- ( ((1/n) * ((X %*% v) / norm_vec(v)) - u )^2 ) / ( 1 - (1/n)* (sum(diag(S)))/(1 + a
  GCV[i] <- GCV_alpha
}

opt.alpha <- alphas_u[which.min(GCV)]
opt_s.alpha <- S_alphas_u[[which.min(GCV)]]
Omega_u <- Omega_u[[which.min(GCV)]]
#close(pb)
result <- list(GCV_u = GCV, opt.alpha_u = opt.alpha, opt_s.alpha_u = opt_s.alpha, GCVdf = data.frame
  return(result)
}
}

GCV_result_u <- opt_alpha_u(X = X_temp,
                           n_var = n_var,
                           ncol = ncol,
                           S_alphas_v = as.matrix(bdiag(GCV_result_v$opt_s.alpha_v)),
                           S_alphas_u = S_alpha_list_u,
                           alphas_u = smooth_tuning_row,
                           alpha_v = GCV_result_v$opt.alpha_v,
                           sparse_tuning_result_u = gamma_u,
                           sparse_tuning_result_v = gamma_v,
                           Omega_v = data.frame(Omegas_v[smooth_tuning_col == GCV_result_v$opt.alpha_v,
                           Omegas_u = Omegas_u,
                           sparse_tuning_type = sparse_tuning_type)

```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length
```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length
```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length
```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length
```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length
```

```
## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
```

```

## replacement length

## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length

## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length

## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length

## Warning in GCV[i] <- GCV_alpha: number of items to replace is not a multiple of
## replacement length

GCV_result_u$opt.alpha

## [1] 3.027727e-06

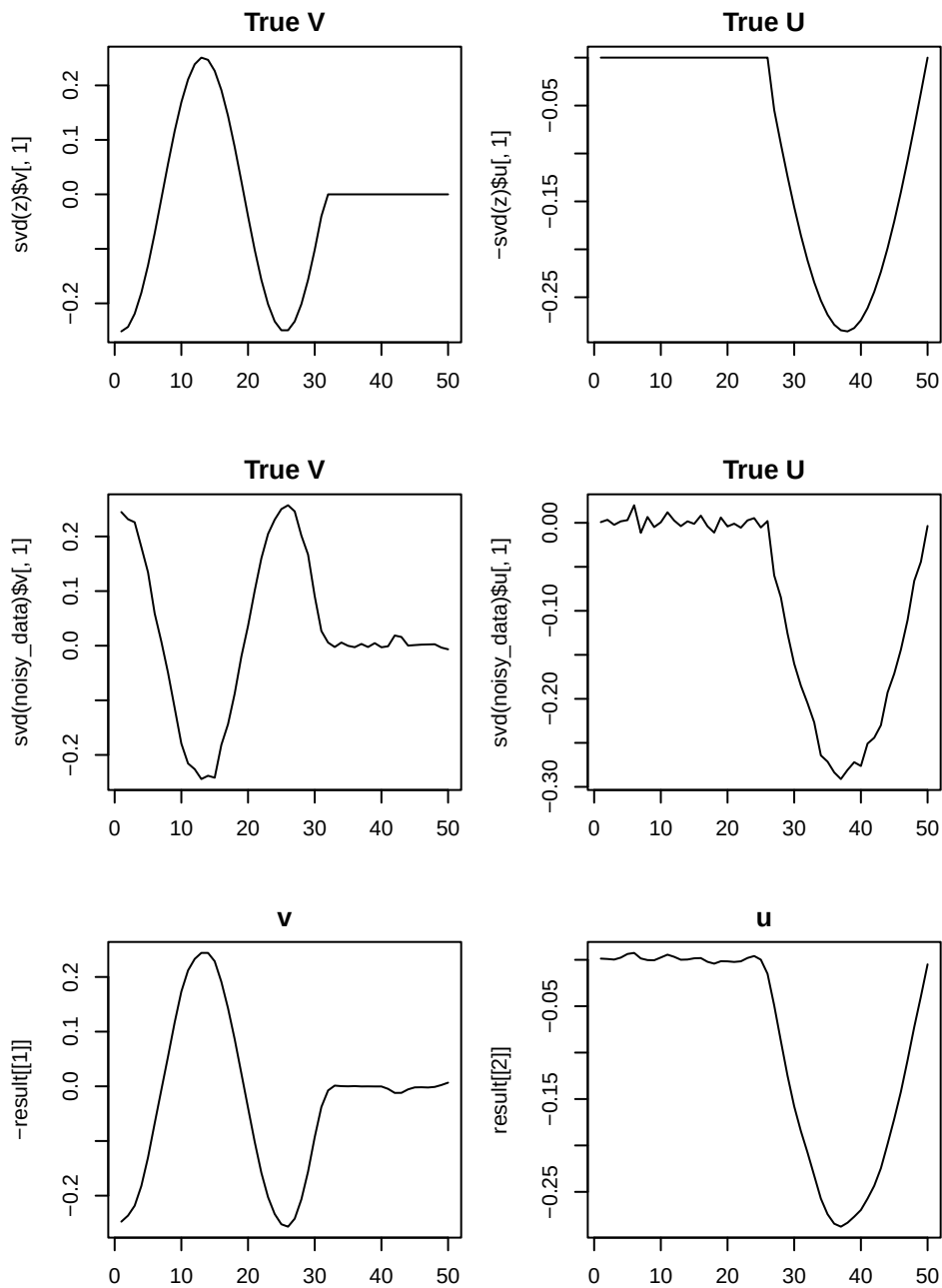
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = 0, # A fixed number
                     sparse_tuning_result_v = c(0,0), # A vector (length = p)
                     S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (length = p)
                     S_alpha_u = GCV_result_u$opt_s.alpha_u, # A matrix
                     sparse_tuning_type = sparse_tuning_type)

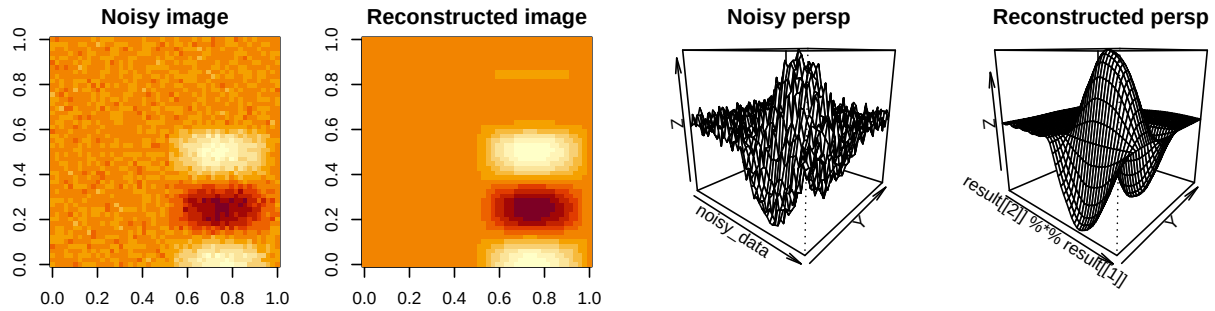
matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```



7. Conditional two-way sparsity two-way smoothness

```

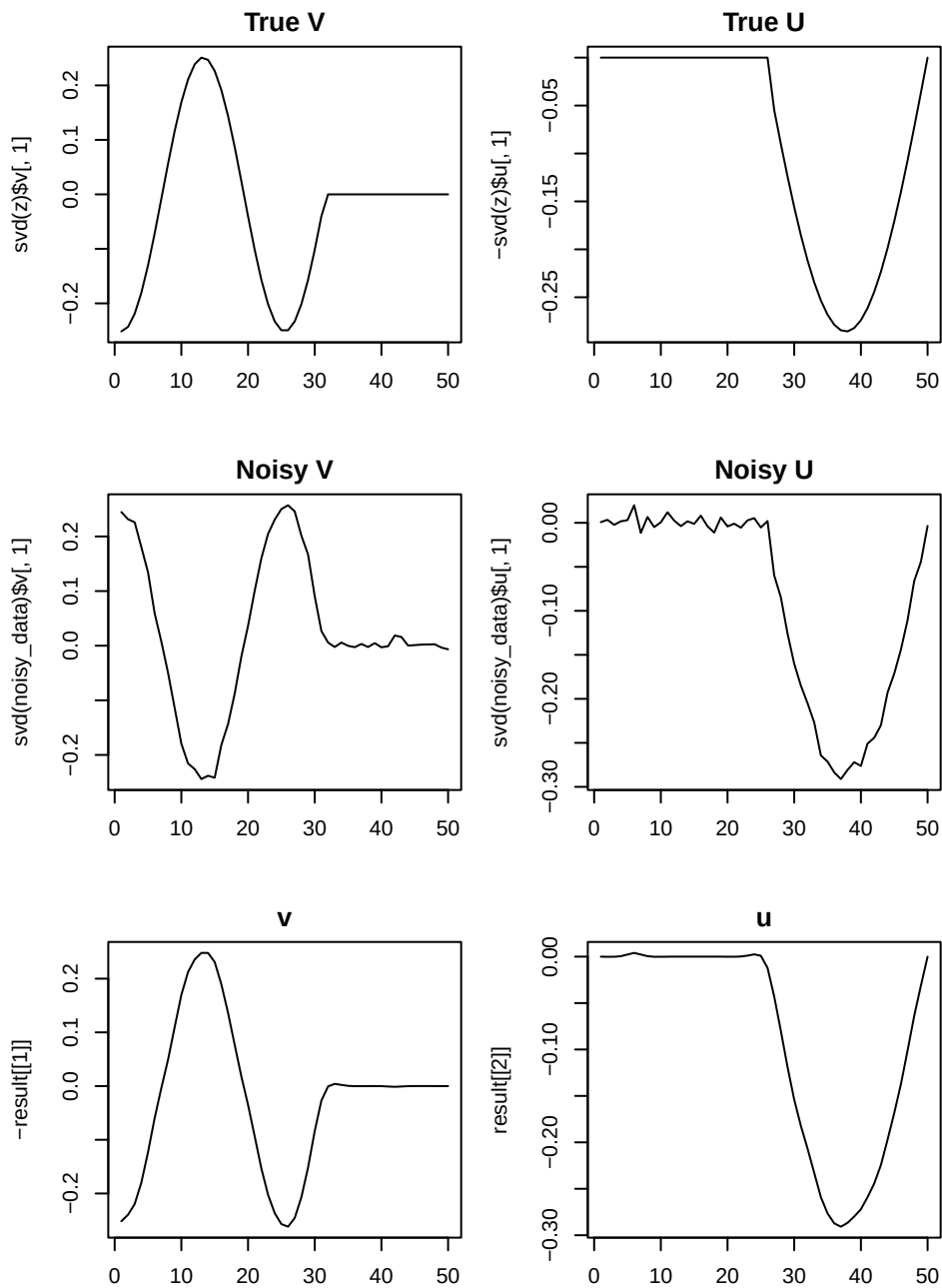
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "Noisy V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "Noisy U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = gamma_u, # A fixed number
                     sparse_tuning_result_v = gamma_v, # A vector (length = p)
                     S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (length = p)
                     S_alpha_u = GCV_result_u$opt_s.alpha_u, # A matrix
                     sparse_tuning_type = sparse_tuning_type)

matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```

```
par(mfrow = c(2,3), mar = c(2, 2, 2, 2))
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")
image(z, main = "TRUE image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
persp(z, theta = 40, main = "TRUE persp")
```

